

# Сравнение FDM и PINN для решения 3D уравнения теплопроводности в JAX

доклад на семинаре

Тимур Фатахов, Сергей Тишковец, Чинь Тхи Тху Хоай, Игорь Кан  
17.04.2026

- Проект: аналитическое решение, FDM-солвер, PINN-солвер и модуль валидации
- Цель: проверить корректность реализации и сравнить методы по точности, устойчивости и вычислительной стоимости
- JAX используется и для JIT-компиляции разностной схемы, и для автоматического дифференцирования PINN

# Постановка задачи

Рассматривается нестационарное уравнение теплопроводности в кубической области.

$$u_t = \alpha(u_{xx} + u_{yy} + u_{zz})$$

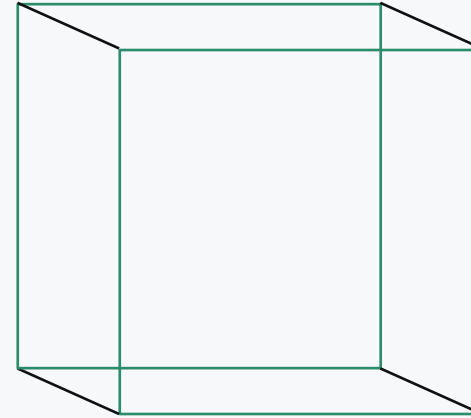
## Область расчёта

$$(x, y, z) \in [0, 1]^3, \quad t \in [0, T]$$

## Начальное условие

$$u(x, y, z, 0) = \sin(\pi x)\sin(\pi y)\sin(\pi z)$$

- Граничные условия Дирихле:  $u = 0$  на всех гранях куба
- Фактическая конфигурация эксперимента:  $\alpha = 0.1, T = 1.0$



$$\Omega = [0, 1]^3$$

Схема области и граничных условий

## Аналитическое решение задачи

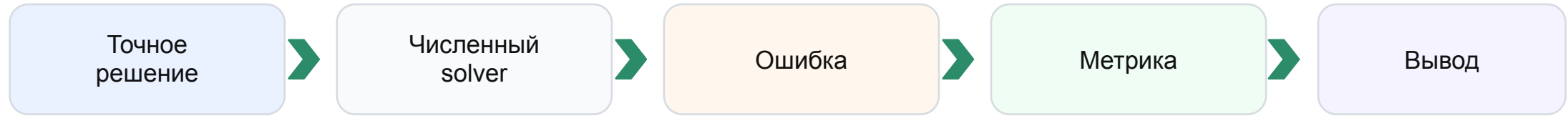
Для выбранного начального условия точное решение имеет вид.

$$u(x, y, z, t) = e^{-3\alpha\pi^2 t} \sin(\pi x) \sin(\pi y) \sin(\pi z)$$

- удовлетворяет исходному уравнению теплопроводности
- точно выполняет начальное условие
- точно удовлетворяет нулевым граничным условиям
- демонстрирует затухание амплитуды со временем
- используется как эталон при вычислении ошибок

# Зачем нужен аналитический benchmark

Наличие точного решения позволяет перейти от демонстрации работы программы к полноценной численной валидации.



- сравнение численного решения с эталоном в каждой точке сетки
- вычисление ошибок  $L_2$ ,  $L^\infty$  и относительной  $L_2$
- проверка сходимости метода при измельчении сетки
- корректное сравнение FDM и PINN на одной физической задаче

В этой работе аналитическое решение используется как единый критерий качества для обоих методов.

# Математическая идея FDM

- Непрерывная задача заменяется системой разностных уравнений на регулярной 3D сетке
- Центральные разности второго порядка аппроксимируют вторые производные по пространству
- По времени применяется явная схема Эйлера

$$u_{i,j,k}^{n+1} = u_{i,j,k}^n + \Delta t \alpha \nabla_h^2 u_{i,j,k}^n$$

Прозрачная численная интерпретация

Предсказуемая устойчивость

Проверка теоретического порядка сходимости

## FDM: лапласиан и аппроксимация второго порядка

Пространственный оператор Лапласа реализован через центральные разности второго порядка по трём координатам.

Во внутренних узлах вычисляются  $u_{xx}$ ,  $u_{yy}$  и  $u_{zz}$ .

$$u_{xx} \approx \frac{u_{i+1,j,k} - 2u_{i,j,k} + u_{i-1,j,k}}{\Delta x^2}$$

- Аналогично вычисляются  $u_{yy}$  и  $u_{zz}$ , после чего они суммируются
- Пространственная аппроксимация имеет ожидаемую точность  $O(\Delta x^2)$
- Далее проверяется второй порядок сходимости FDM

## FDM: код оператора Лапласа

Код реализует вычисление трёхмерного лапласиана только во внутренних узлах сетки.  
Реализация полностью векторизована через срезы массивов JAX.

src/fdm\_solver.py

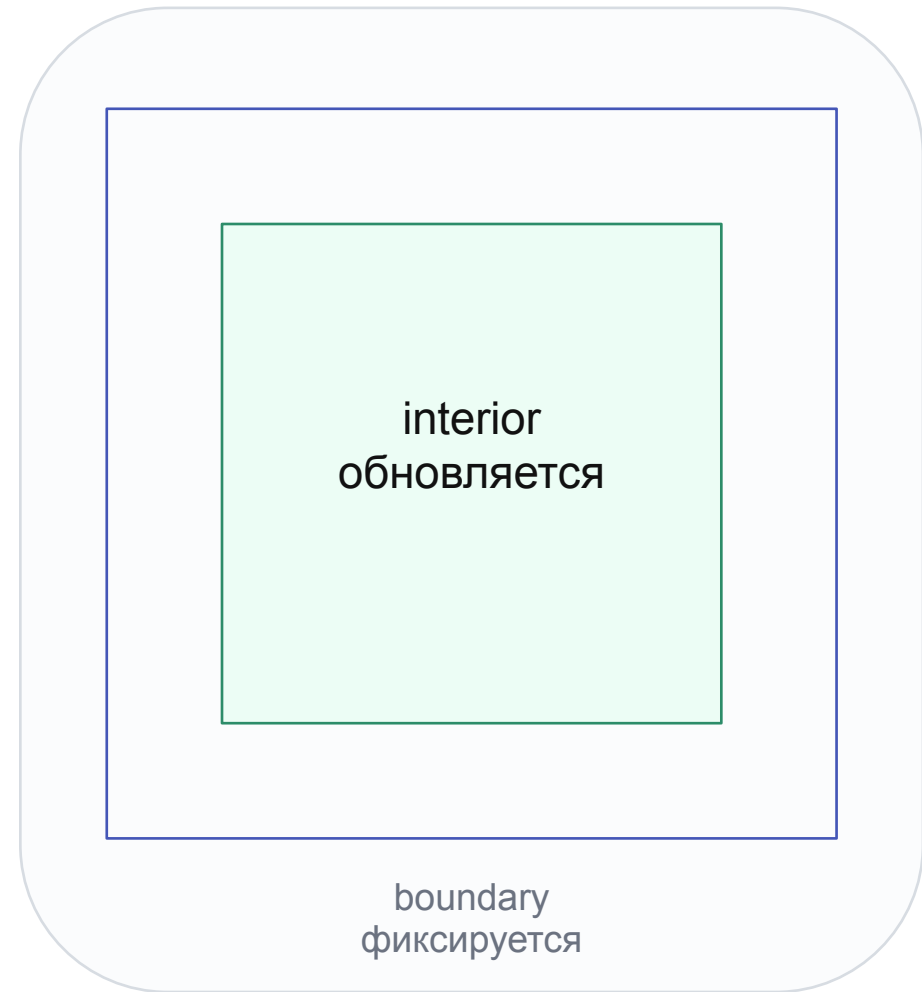
```
1 @jit
2 def laplacian_3d(u: jnp.ndarray, dx: float, dy: float, dz: float) -> jnp.ndarray:
3     d2uxx = (u[2:, 1:-1, 1:-1] - 2.0 * u[1:-1, 1:-1, 1:-1]
4             + u[:-2, 1:-1, 1:-1]) / dx**2
5     d2uyy = (u[1:-1, 2:, 1:-1] - 2.0 * u[1:-1, 1:-1, 1:-1]
6             + u[1:-1, :-2, 1:-1]) / dy**2
7     d2uzz = (u[1:-1, 1:-1, 2:] - 2.0 * u[1:-1, 1:-1, 1:-1]
8             + u[1:-1, 1:-1, :-2]) / dz**2
9     return d2uxx + d2uyy + d2uzz
```

## FDM: шаг Эйлера и граничные условия

Один шаг по времени реализован явной схемой Эйлера.

$$u_{\text{int}}^{n+1} = u_{\text{int}}^n + \alpha \Delta t \nabla_h^2 u^n$$

- обновляется только внутренняя часть массива
- граничные узлы не пересчитываются
- нулевые граничные условия Дирихле сохраняются автоматически
- математическая постановка напрямую связана с кодом солвера



## FDM: код временного шага

Функция `euler_step` вычисляет лапласиан и обновляет только внутренние узлы сетки.

```
src/fdm_solver.py
1 @jit
2 def euler_step(u: jnp.ndarray, dt: float, alpha: float,
3               dx: float, dy: float, dz: float) -> jnp.ndarray:
4     lap = laplacian_3d(u, dx, dy, dz)
5     u_next_interior = u[1:-1, 1:-1, 1:-1] + alpha * dt * lap
6     return u.at[1:-1, 1:-1, 1:-1].set(u_next_interior)
```

Операция `.at[1:-1, 1:-1, 1:-1].set(...)` явно фиксирует область обновления; границы остаются нулевыми без отдельного шага постобработки.

# FDM: устойчивость и условие CFL

Для явной схемы Эйлера в 3D используется ограничение на шаг по времени.

$$dt_{\max} = \frac{\min(dx, dy, dz)^2}{6\alpha}$$

- вычисляются шаги сетки dx, dy, dz
- оценивается допустимый dt\_max
- если dt больше допустимого, шаг автоматически уменьшается до  $0.95 \cdot dt_{\max}$
- это предотвращает запуск явной схемы в неустойчивом режиме
- это необходимо для контроля численной устойчивости

контроль устойчивости

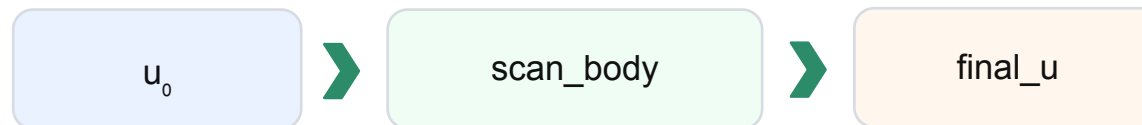
```
1 dx = Lx / (Nx - 1)
2 dy = Ly / (Ny - 1)
3 dz = Lz / (Nz - 1)
4
5 dt_max = min(dx, dy, dz)**2 / (6.0 * alpha)
6 if dt > dt_max:
7     print(f"ВНИМАНИЕ: dt = {dt:.2e} > dt_max = {dt_max:.2e}")
8     dt_used = dt_max * 0.95
9     print(f"dt автоматически скорректирован до {dt_used:.2e}")
```

## FDM: временной цикл в JAX через lax.scan

- После выбора шага по времени решатель строит начальное условие и выполняет интегрирование по времени
- Вместо обычного Python-цикла используется `jax.lax.scan`
- Это делает реализацию JIT-совместимой и отделяет численный шаг от механики временного цикла

временной цикл

```
1 n_steps = int(round(T / dt_used))
2 if n_steps < 1:
3     n_steps = 1
4
5 u0 = create_initial_condition(Nx, Ny, Nz, Lx, Ly, Lz)
6
7 def scan_body(carry, _):
8     return euler_step(carry, dt_used, alpha, dx, dy, dz),
9     None
10 final_u, _ = lax.scan(scan_body, u0, None, length=n_steps)
```



## FDM: схема валидации и численные метрики

Для FDM решение сравнивается с аналитическим эталоном на последовательности сеток.

$11^3$ ,  $21^3$ ,  $31^3$ ,  $41^3$

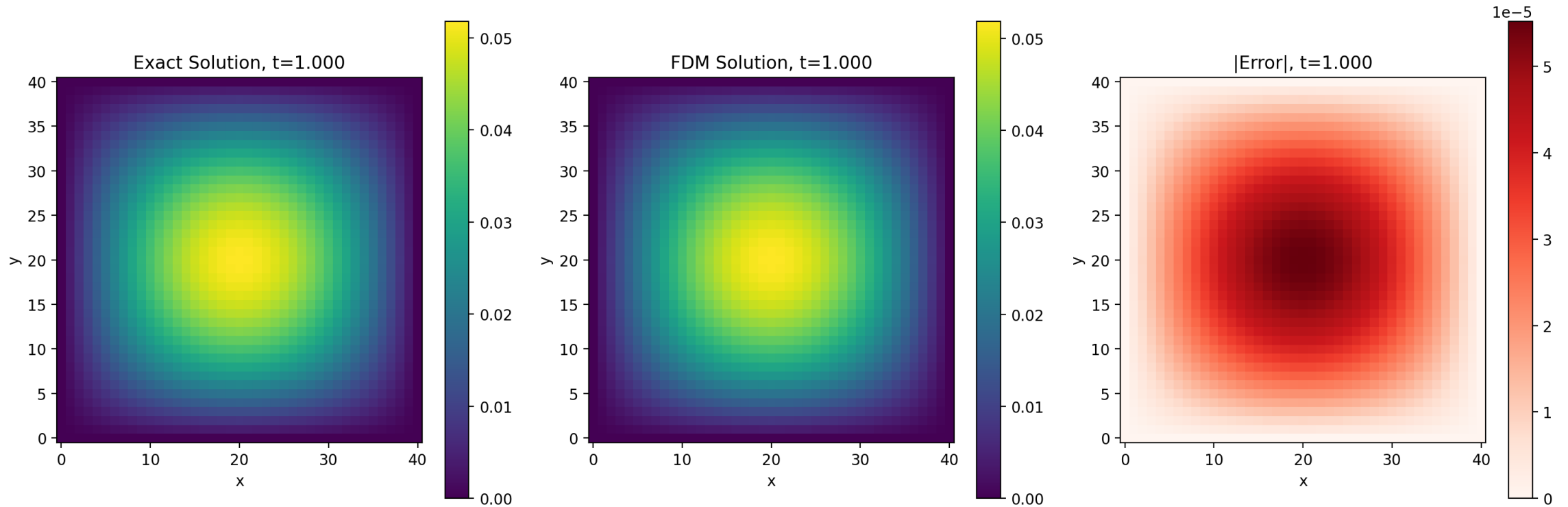
- для каждой сетки вычисляются абсолютные ошибки  $L_2$  и  $L_\infty$ , относительная ошибка  $L_2$  и шаг по времени  $dt$
- при измельчении сетки ошибки монотонно уменьшаются

| Сетка  | $L_2$    | $L_\infty$ | rel $L_2$ | $dt$     |
|--------|----------|------------|-----------|----------|
| $11^3$ | 2.73e-04 | 8.91e-04   | 5.27e-03  | 1.67E-03 |
| $21^3$ | 7.27e-05 | 2.21e-04   | 1.40e-03  | 4.17E-04 |
| $31^3$ | 3.30e-05 | 9.82e-05   | 6.38e-04  | 1.85E-04 |
| $41^3$ | 1.88e-05 | 5.52e-05   | 3.63e-04  | 1.04E-04 |

Наблюдение: при измельчении сетки ошибки уменьшаются монотонно, что согласуется с корректной реализацией схемы.

## FDM: визуальное сравнение с точным решением

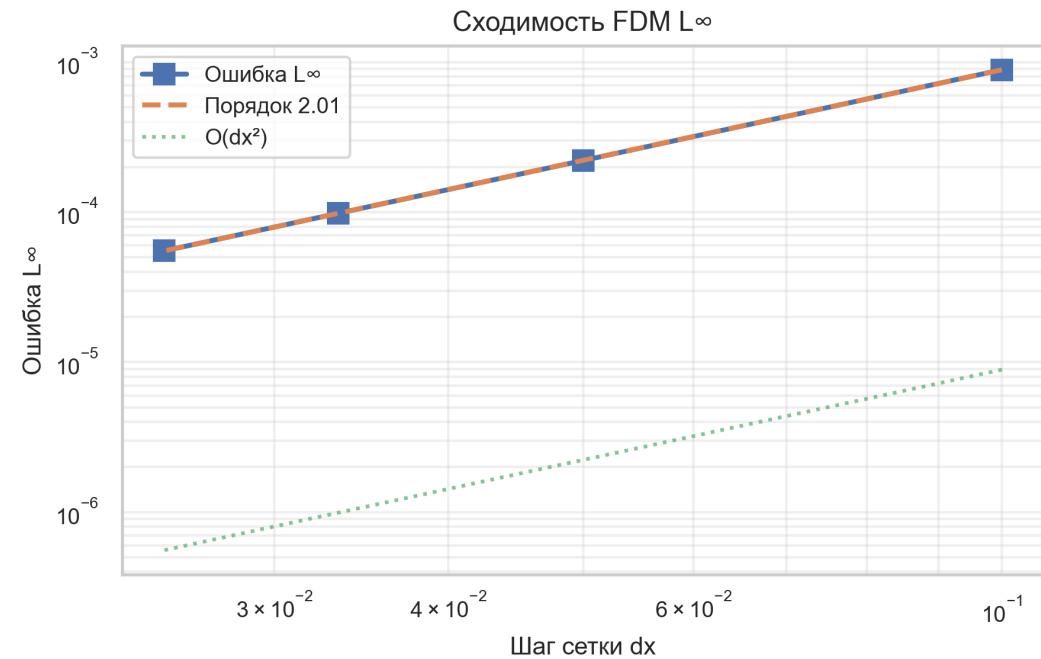
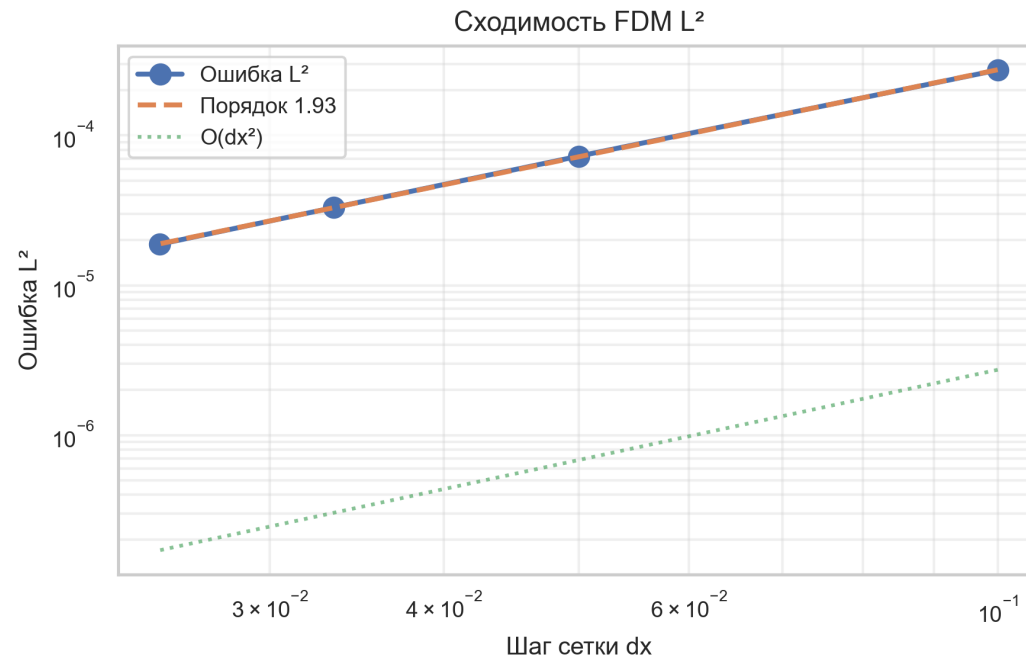
- Сравниваются точное решение, численное решение FDM и карта абсолютной ошибки для  $t = 1.0$
- Форма численного решения практически совпадает с аналитическим эталоном
- Ошибка мала на всей области и не искажает структуру решения



# FDM: исследование сходимости

- На логарифмическом графике показано уменьшение ошибки FDM при уменьшении шага сетки
- Полученные оценки практически совпадают с теоретическим вторым порядком
- Это подтверждает корректную реализацию центральных разностей второго порядка

$$p_{L_2} = 1.93, \quad p_{\text{rel}, L_2} = 1.93, \quad p_{L_\infty} = 2.01$$



# Математическая идея PINN

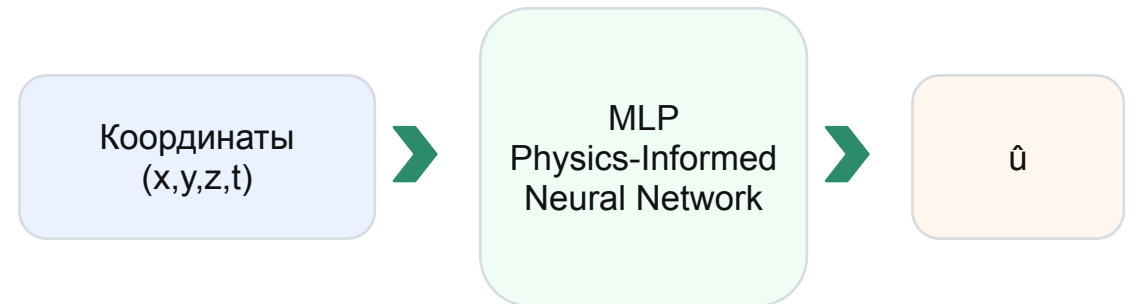
В PINN решение представляется нейросетевой аппроксимацией.

$$u_{\theta}(x, y, z, t)$$

На вход сети подаются координаты и время

$$(x, y, z, t)$$

- сеть обучается минимизировать невязку исходного уравнения
- одновременно учитываются начальные и граничные условия
- производные  $u_t$ ,  $u_{xx}$ ,  $u_{yy}$ ,  $u_{zz}$  вычисляются через automatic differentiation JAX
- решение задаётся как непрерывная аппроксимация функции в пространстве  $(x, y, z, t)$



# PINN: архитектура сети и гиперпараметры

## Архитектура

- вход: 4 переменные (x,y,z,t)
- скрытых слоёв: 5
- ширина слоя: 96
- выход: 1 значение  $u(x,y,z,t)$
- эквивалентная схема:  
[4,96,96,96,96,96,1]

[4, 96, 96, 96, 96, 96, 1]

tanh на скрытых слоях,  
линейный выход

| Параметр                                          | Значение       |
|---------------------------------------------------|----------------|
| epochs                                            | 8000           |
| learning_rate                                     | 0.001 → 0.0001 |
| n_residual                                        | 12000          |
| n_boundary                                        | 2000           |
| n_initial                                         | 2000           |
| $\lambda_{pde}$ / $\lambda_{bc}$ / $\lambda_{ic}$ | 1 / 10 / 10    |
| hard constraints                                  | true           |
| eval_grid_size                                    | 21             |

## PINN: код архитектуры MLP

Показаны инициализация параметров MLP и прямой проход сети.  
Скрытые слои используют tanh, выходной слой - линейный.

```
src/pinn_solver.py
1 def init_mlp_params(layer_sizes, key) -> list[dict[str, Array]]:
2     keys = jax.random.split(key, len(layer_sizes) - 1)
3     params = []
4     for k, (in_dim, out_dim) in zip(keys, zip(layer_sizes[:-1], layer_sizes[1:])):
5         W = _glorot_std(in_dim, out_dim) * jax.random.normal(k, (in_dim, out_dim))
6         b = jnp.zeros((out_dim,))
7         params.append({"W": W, "b": b})
8     return params
9
10 def forward_mlp(params: list[dict[str, Array]], x: Array) -> Array:
11     h = x
12     for layer in params[:-1]:
13         h = jnp.tanh(h @ layer["W"] + layer["b"])
14     return h @ params[-1]["W"] + params[-1]["b"]
```

## PINN: жёсткое наложение условий и преобразование выхода

В данной реализации начальные и граничные условия не только учитываются в **функции потерь**, но и встраиваются непосредственно в форму решения.

$$u(x, y, z, t) = (1 - \tau)u_0 + \tau \cdot env \cdot raw_{out}$$

- $u_0$  - точное начальное условие
- $\tau$  - временной множитель, равный нулю при  $t = 0$
- $env$  - пространственная оболочка, обращающаяся в ноль на границе
- $raw_{out}$  - необработанный выход нейросети
- следствие: при  $t = 0$  выполняется  $u = u_0$ , а на границе -  $u = 0$

## PINN: код output transform

Hard constraints встроены непосредственно в вычисление предсказания сети.

src/pinn\_solver.py

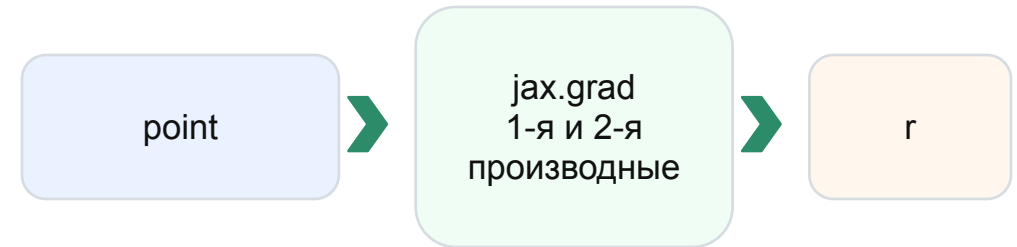
```
1 def apply_output_transform(raw_out: Array, points: Array, cfg: PINNConfig) -> Array:
2     u0 = initial_condition(points, cfg)
3     tau = time_factor(points, cfg)
4     env = boundary_envelope(points, cfg)
5     return (1.0 - tau) * u0 + tau * env * raw_out
6
7 def predict(params: list[dict[str, Array]], points: Array, cfg: PINNConfig) -> Array:
8     raw = forward_mlp(params, normalize_inputs(points, cfg))
9     if cfg.use_hard_constraints:
10         return apply_output_transform(raw, points, cfg)
11     return raw
```

# PINN: PDE residual через автоматическое дифференцирование

PINN обучается по невязке исходного уравнения теплопроводности.

$$r = u_t - \alpha(u_{xx} + u_{yy} + u_{zz})$$

- $u_t$  вычисляется через `jax.grad`
- вторые производные по пространству получаются через вложенные вызовы `jax.grad`
- из производных формируется невязка PDE в каждой collocation-точке
- сеть обучается не только по значениям, а по выполнению физического уравнения



## PINN: код вычисления PDE residual

Ниже показано вычисление производных и формирование невязки уравнения.

src/pinn\_solver.py

```
1 def pde_residual_single(params: list[dict[str, Array]], point: Array, cfg: PINNConfig) -> Array:
2     grad_u = jax.grad(_scalar_predict, argnums=1)(params, point, cfg)
3     u_t = grad_u[3]
4
5     def du_dx(_point):
6         return jax.grad(_scalar_predict, argnums=1)(params, _point, cfg)[0]
7
8     def du_dy(_point):
9         return jax.grad(_scalar_predict, argnums=1)(params, _point, cfg)[1]
10
11    def du_dz(_point):
12        return jax.grad(_scalar_predict, argnums=1)(params, _point, cfg)[2]
13
14    u_xx = jax.grad(du_dx)(point)[0]
15    u_yy = jax.grad(du_dy)(point)[1]
16    u_zz = jax.grad(du_dz)(point)[2]
17    return u_t - cfg.alpha * (u_xx + u_yy + u_zz)
```

## PINN: функция потерь

Общая функция потерь складывается из PDE-компоненты и компонентов, связанных с граничными и начальными условиями.

$$\mathcal{L} = \lambda_{pde}\mathcal{L}_{pde} + \lambda_{bc}\mathcal{L}_{bc} + \lambda_{ic}\mathcal{L}_{ic}$$

- `loss_pde` - квадрат невязки PDE
- `loss_bc` - ошибка выполнения граничных условий
- `loss_ic` - ошибка выполнения начального условия
- при `use_hard_constraints = true` компоненты BC и IC становятся несущественными

```
loss_terms
1 def loss_terms(params, batch: Dict[str,
Array], cfg: PINNConfig):
2     r = pde_residual_batch(params,
batch["x_r"], cfg)
3     loss_pde = jnp.mean(r ** 2)
4
5     if cfg.use_hard_constraints:
6         loss_bc = jnp.array(0.0)
7         loss_ic = jnp.array(0.0)
8     else:
9         pred_b = predict(params,
batch["x_b"], cfg)
10        loss_bc = jnp.mean((pred_b -
batch["y_b"]) ** 2)
11        pred_i = predict(params,
batch["x_i"], cfg)
12        loss_ic = jnp.mean((pred_i -
batch["y_i"]) ** 2)
```

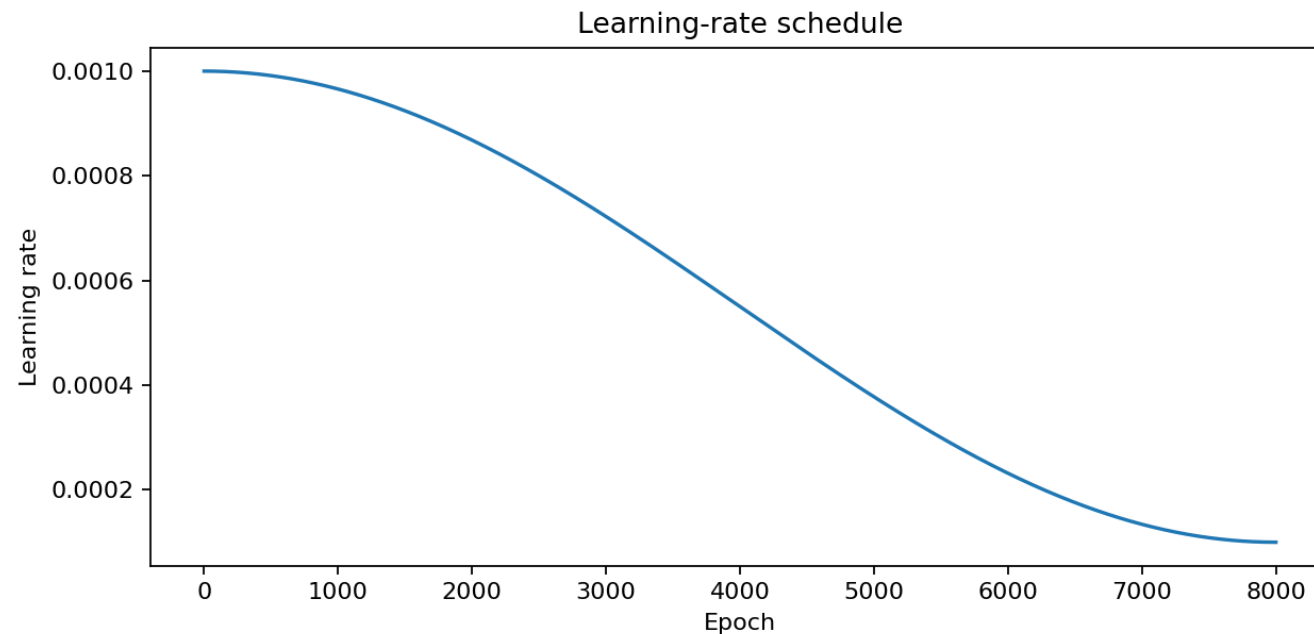
## PINN: цикл обучения

- Процедура обучения включает генерацию batch, шаг оптимизатора и логирование learning rate и loss-компонент
- После обучения вычисляются метрики на контрольных временах eval\_times
- На каждой эпохе сохраняются: loss\_total, loss\_pde, loss\_bc, loss\_ic, learning\_rate

```
train_pinn
1 def train_pinn(cfg: PINNConfig | None = None):
2     cfg = cfg or PINNConfig()
3     params, optimizer, opt_state = create_train_state(cfg)
4     key = jax.random.PRNGKey(cfg.seed + 123)
5
6     total_epochs = cfg.pretrain_ic_bc_epochs + cfg.epochs
7     for global_epoch in range(1, total_epochs + 1):
8         key, subkey = jax.random.split(key)
9         batch = make_training_batch(subkey, cfg)
10        if global_epoch <= cfg.pretrain_ic_bc_epochs:
11            lr = cfg.learning_rate
12        else:
13            lr = cosine_decay_lr(global_epoch - cfg.pretrain_ic_bc_epochs, cfg)
14        params, opt_state, metrics = train_step(params, opt_state, batch, optimizer, cfg, lr, "full")
15
16        metrics_by_time = [compute_error_metrics(params, cfg, t) for t in cfg.eval_times]
17        train_time_s = float(time.perf_counter() - start_time)
18        history["train_time_s"] = train_time_s
19        history["training_time_sec"] = train_time_s
20    return params, history, metrics_by_time, cfg, train_time_s
```

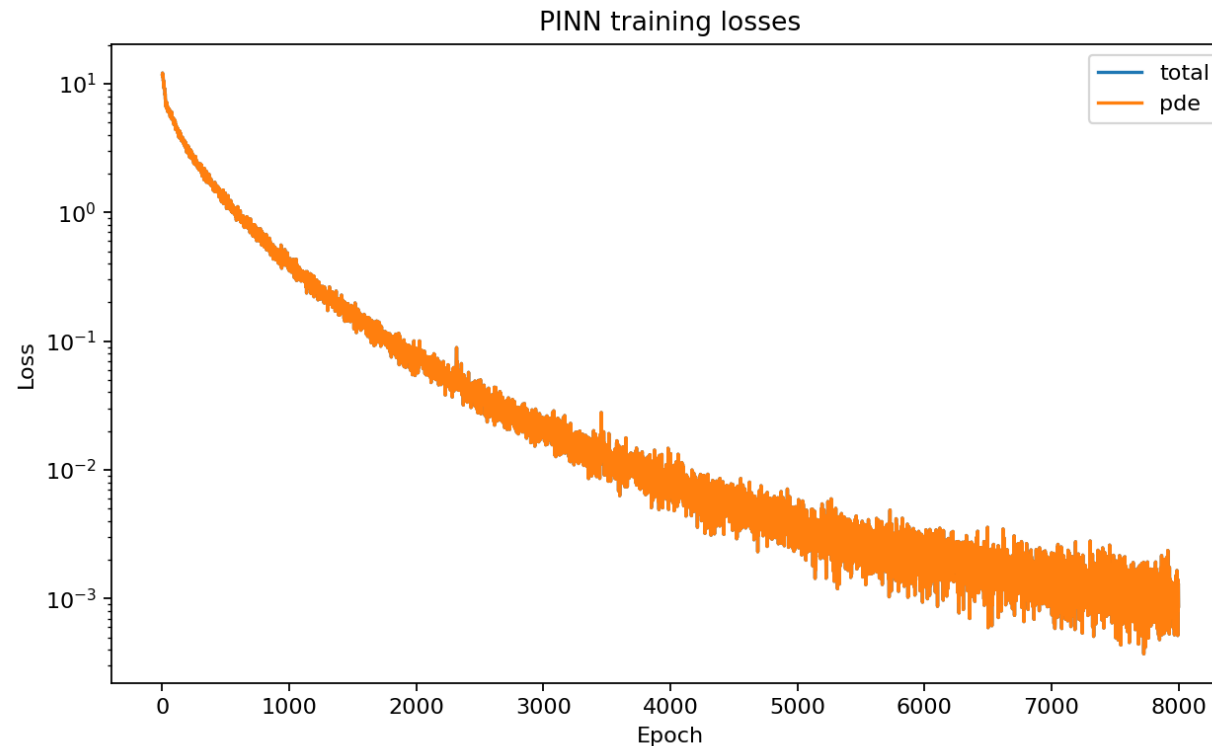
## PINN: расписание скорости обучения

- Во время обучения используется убывающая скорость обучения: от 0.001 до 0.0001
- На ранних этапах шаг больше для быстрой подстройки параметров
- На поздних этапах шаг уменьшается для стабилизации оптимизации
- Scheduler влияет на устойчивость обучения PINN



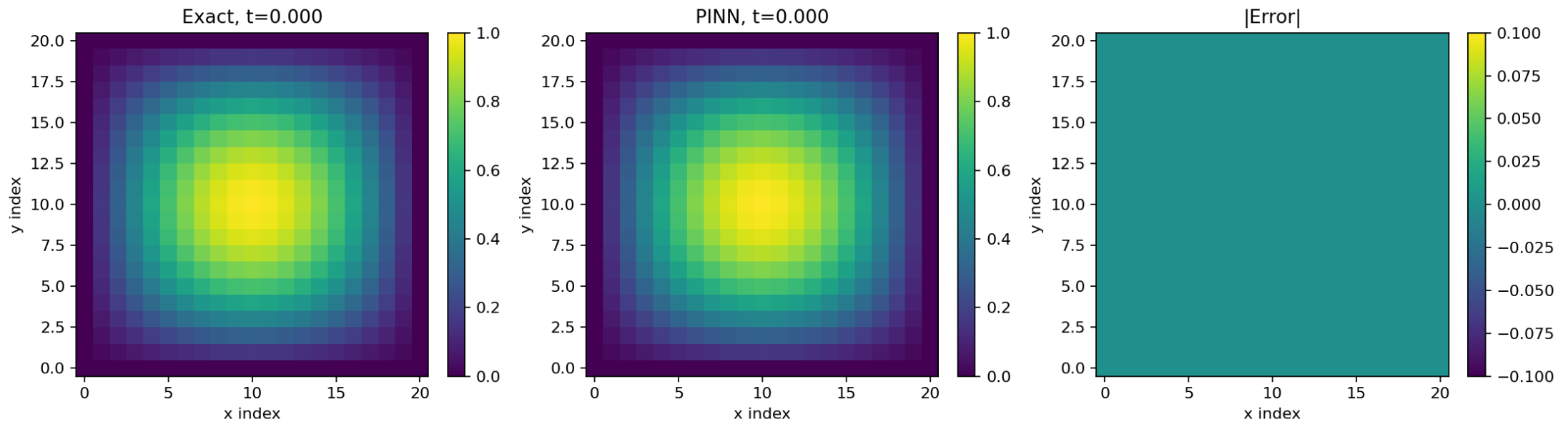
## PINN: история функции потерь

- На графике показана динамика общей потери total и PDE-компоненты в логарифмической шкале
- loss уменьшается примерно с  $10^1$  до  $10^{-3}$
- Кривые total и pde почти совпадают, что согласуется с hard constraints
- К концу обучения сеть выходит на устойчивый режим



## PINN: сравнение exact и prediction на $t = 0$

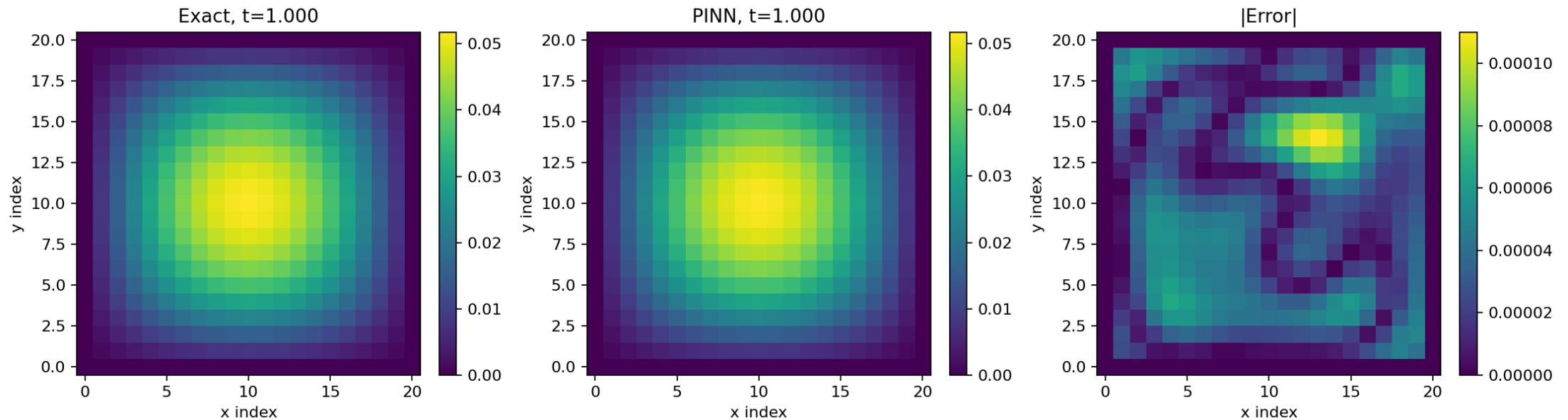
- Показаны точное решение, решение PINN и карта абсолютной ошибки для начального момента времени
- PINN практически точно воспроизводит начальное условие
- Это прямое подтверждение корректной работы output transform и hard constraints



## PINN: сравнение exact и prediction на $t = 1.0$

- Показаны точное решение, решение PINN и карта абсолютной ошибки для финального момента рассматриваемого интервала
- PINN сохраняет корректную форму решения, но ошибка возрастает по сравнению с более ранними слоями
- Качество остаётся приемлемым, но заметно уступает FDM на этой задаче

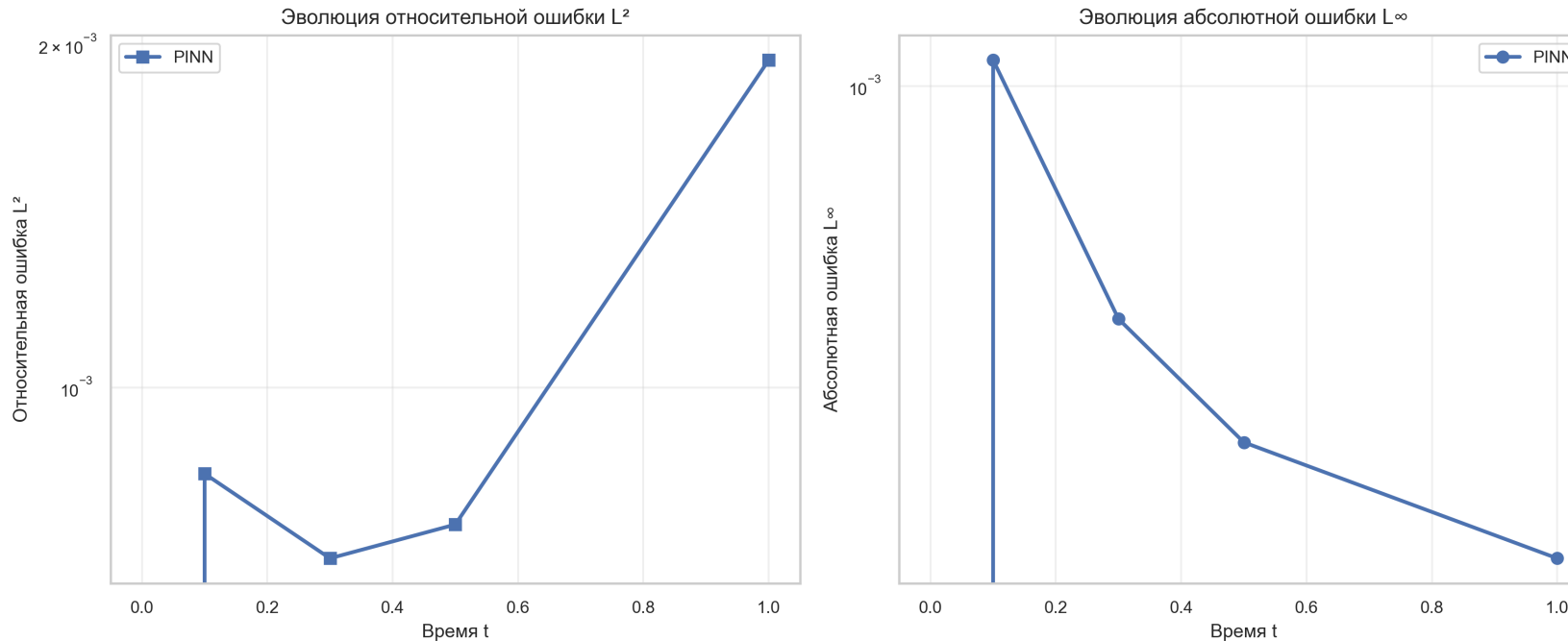
$$L_2 = 2.4063 \times 10^{-4} \quad | \quad L_\infty = 1.18 \times 10^{-4} \quad | \quad \text{rel } L_2 = 1.94 \times 10^{-3}$$



# PINN: эволюция ошибки по времени

На графике показана динамика относительной ошибки  $L_2$  и абсолютной ошибки  $L_\infty$ .

- $t = 0.10$ :  $\text{rel } L_2 = 8.40 \times 10^{-4}$ ,  $L_\infty = 1.12 \times 10^{-3}$
- $t = 0.30$ :  $\text{rel } L_2 = 7.08 \times 10^{-4}$ ,  $L_\infty = 3.49 \times 10^{-4}$
- $t = 0.50$ :  $\text{rel } L_2 = 7.58 \times 10^{-4}$ ,  $L_\infty = 2.00 \times 10^{-4}$
- $t = 1.00$ :  $\text{rel } L_2 = 1.94 \times 10^{-3}$ ,  $L_\infty = 1.18 \times 10^{-4}$
- Относительная ошибка остаётся порядка  $10^{-3}$  на всём интервале; к финальному моменту  $\text{rel } L_2$  возрастает, а абсолютная  $L_\infty$  уменьшается.



## Итоговое сравнение FDM и PINN

| Метод | Сетка  | L2 / rel L2 | $L^\infty$ | Время                                |
|-------|--------|-------------|------------|--------------------------------------|
| FDM   | $21^3$ | 7.27E-05    | 2.21E-04   | 1.98 s                               |
| PINN  | $21^3$ | 1.94E-03    | 1.18E-04   | обучение 14177.9 s, inference 0.03 s |

### Основные выводы

- FDM на данной аналитической задаче существенно точнее PINN
- FDM демонстрирует подтверждённый второй порядок сходимости
- FDM решает задачу быстро и предсказуемо
- PINN даёт корректное решение, но требует длительного обучения

### Ограничения сравнения

- PINN требует длительного обучения
- Для PINN существуют ограничения по памяти при переходе к более плотным 3D сеткам
- FDM и PINN теперь сравниваются на одной сетке  $21^3$ .

FDM остаётся более практичным методом для данной гладкой задачи: он быстрее получает решение и даёт меньшую L2-ошибку. PINN показывает малое время инференса после обучения, но само обучение существенно дороже.

## Список литературы

1. Raissi, M., Perdikaris, P., Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019. ([ScienceDirect](#))
2. LeVeque, R. J. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007. ([SIAM E-Books](#))
3. Strang, G. *Computational Science and Engineering*. Wellesley-Cambridge Press, 2007. ([ScienceDirect](#))
4. JAX Developers. *JAX Documentation*. Official documentation, 2026. ([JAX Documentation](#))
5. JAX Developers. *Quickstart: How to think in JAX*. Official documentation, 2026. ([JAX Documentation](#))
6. JAX Developers. *API Reference*. Official documentation, 2026. ([JAX Documentation](#))